

A multilevel $\mathcal{O}(N \log N)$ algorithm for evaluating the attenuated Radon transform

Oren E. Livne*

Achi Brandt†

September 27, 2026

Abstract

The forward operator of emission tomography is the attenuated Radon transform, in which the attenuation enters through an exponential whose argument varies along the line of integration. Evaluating it on an n^d grid for all significantly different lines costs $\mathcal{O}(n^{2d-1})$ operations by direct quadrature — one factor of n for each of the $2(d-1)$ parameters of a line, and one more for traversing it — and this evaluation is the inner loop of every iterative reconstruction method. We give an algorithm that removes the traversal factor, costing $\mathcal{O}(N \log N \log(1/\varepsilon))$ in two dimensions for a prescribed accuracy ε , and in three dimensions the size of the output itself, by carrying the recursive length-doubling construction of Brandt and Dym for plain line integrals over to the attenuated case. Two ingredients make the transfer work. An exact concatenation identity for a triple of segment integrals lets the integration length double from level to level with no approximation. And indexing segments by a starting point on a fixed mesh together with an integer rise — so that directions are equispaced in slope and every segment begins and ends on the mesh — means that doubling needs no spatial interpolation at all, only a one-dimensional interpolation in the slope index, at $\mathcal{O}(p)$ operations per integral for an order- p stencil. We identify the quantities that must be interpolated: the attenuation exponent rather than its exponential, a segment integral referenced to its own exit point rather than to its midpoint, and both normalised by arc length. Because the attenuated merge is nonlinear the interpolation must precede the concatenation, which is not so for a plain line integral. Constant data is then reproduced exactly at every level, in two and three dimensions, to 4×10^{-16} . The four-quadrant transform of a 129^2 image takes 0.63 s, and the full four-dimensional transform of a 33^3 image — 2.8×10^7 line integrals — takes 43 s.

1 Introduction

1.1 The forward model

Emission tomography reconstructs the spatial distribution of a radiotracer from counts of the photons it emits. Two fields are involved: the activity $f(\mathbf{x}) \geq 0$, which is the unknown, and the linear attenuation coefficient $\mu(\mathbf{x}) \geq 0$ of the surrounding medium at the photon energy, which is usually measured separately by a transmission or CT scan.

Parameterise a line in $\mathbb{R}^2$ by a direction and a signed offset,

$$\boldsymbol{\omega}(\theta) = (\cos \theta, \sin \theta), \quad \boldsymbol{\omega}^\perp(\theta) = (-\sin \theta, \cos \theta), \quad \mathbf{x}(s, t) = s \boldsymbol{\omega}^\perp + t \boldsymbol{\omega}, \quad (1)$$

*Pine Birch Analytics, 35 Kelinger Rd, Churchville, PA 18966-1033, USA (oren.livne@gmail.com, +1 312 533 9130). Corresponding author.

†Department of Computer Science and Applied Mathematics, The Weizmann Institute of Science, Rehovot 7610001, Israel (achi.brandt@weizmann.ac.il).

and traverse $L(\theta, s) = \{\mathbf{x}(s, t) : t \in \mathbb{R}\}$ in the direction of increasing t . A photon emitted at $\mathbf{x}(s, t)$ and travelling along ω escapes with probability $\exp(-\int_t^\infty \mu)$, so the expected count along $L(\theta, s)$ is

$$(\mathcal{R}_\mu f)(\theta, s) = \int_{\mathbb{R}} f(\mathbf{x}(s, t)) \exp\left(-\int_t^\infty \mu(\mathbf{x}(s, \tau)) d\tau\right) dt. \quad (2)$$

This is the attenuated Radon transform, also called the attenuated X-ray transform. Its distinguishing feature is that the exponent depends on the emission point t , so the attenuation does not factor out of the integral. Equation (2) is the forward model of single-photon emission computed tomography (SPECT).

Positron emission tomography (PET) detects two collinear annihilation photons in coincidence. Both must escape, in opposite directions, so the survival probability is $\exp(-\int_t^\infty \mu) \exp(-\int_{-\infty}^t \mu) = \exp(-\int_L \mu)$, independent of t , and

$$(\mathcal{P}_\mu f)(\theta, s) = \exp\left(-\int_L \mu\right) \int_L f. \quad (3)$$

Classical PET attenuation is thus a multiplicative factor per line. The algorithm below computes the three line integrals

$$S = \int_L f, \quad E = \int_L \mu, \quad I = (\mathcal{R}_\mu f)(\theta, s), \quad (4)$$

and therefore delivers both models: (2) is I , and (3) is $e^{-E} S$. The exponent is genuinely position-dependent only in (2), and that is the case treated here; (3) is recovered as a by-product at no extra cost. Time-of-flight PET, in which the coincidence is localised along the line and the two one-sided attenuations no longer combine into a constant, has the structure of (2) and is covered by the same machinery.

1.2 The inverse problem and where evaluation enters

With μ known, (2) is a linear Fredholm integral equation of the first kind for f ,

$$\mathcal{R}_\mu f = g, \quad (5)$$

where g is the measured sinogram. Novikov [14] gave an explicit inversion formula for $\mathcal{R}_\mu$, and for the unattenuated case $\mu \equiv 0$ the classical filtered-backprojection inversion of the Radon transform applies [13]. In practice (5) is solved iteratively rather than by an inversion formula, because the data are Poisson counts, because the system is large and ill-conditioned, and because regularisation and physical corrections are easier to impose inside an iteration.

The standard methods are all of this kind. The expectation-maximisation algorithm for the Poisson likelihood [16] updates

$$f^{(k+1)} = \frac{f^{(k)}}{\mathcal{R}_\mu^* \mathbf{1}} \mathcal{R}_\mu^* \left(\frac{g}{\mathcal{R}_\mu f^{(k)}} \right), \quad (6)$$

a multiplicative — hence nonlinear — relaxation; its ordered-subset variant [10] applies (6) over subsets of directions in sequence, which makes it a block row-action scheme in the family of algebraic reconstruction techniques [8], i.e. a Kaczmarz relaxation with a nonlinear update. Every sweep of every one of these methods applies $\mathcal{R}_\mu$ once and its adjoint once, so the cost of the reconstruction is a small multiple of the cost of evaluating $\mathcal{R}_\mu$.

The exponent alone understates what is at stake. Direct quadrature costs $\mathcal{O}(n^{2d-1})$ and the algorithm below costs $\mathcal{O}(n^{2d-2} \log n)$ in two dimensions and $\mathcal{O}(n^{2d-2})$ in three, so the saving is not the gap between one power of N and another but a single factor of n — the number of samples taken along each line — reduced to $\mathcal{O}(\log n)$ or removed. At $n = 512$ that factor is 512, and it multiplies the number of sweeps.

That the projector and its adjoint dominate iterative reconstruction in practice is not in dispute. On the HRRT brain scanner a CPU reconstruction took days, which kept a high-sensitivity instrument out of routine use until a GPU implementation brought one OSEM iteration with sixteen subsets to 16s, a speed-up of about 125 [11]; and projection is enough of the cost that it is maintained as a separate, dedicated library [15]. Both are responses in hardware to a cost that is algorithmic.

A multilevel evaluation is worth more here than any fast evaluation, because the solver wants the same hierarchy. A row-action relaxation of (5) is a smoother in the multigrid sense, and if μ is estimated jointly with f instead of being measured separately the problem is nonlinear in μ , calling for a full approximation scheme [3]. The level structure built below — segment lengths paired with angular resolutions — is the hierarchy such a solver would traverse. This paper stops at the forward evaluation; the adjoint and the inversion are subsequent work (Section 7).

1.3 Related work and contribution

Fast summation by separating a smooth kernel on a hierarchy of grids is the multilevel matrix-multiplication framework of Brandt and Lubrecht [4, 6]; the fast multipole method [9] decomposes differently. For line integrals, Brandt and Dym [5] gave an $\mathcal{O}(N \log N)$ algorithm based on recursive doubling of segment length, on which Brandt et al. [7] built a multilevel inversion of the unattenuated Radon transform. Hierarchical backprojection at $\mathcal{O}(N \log^2 N)$ [2] and pseudopolar transforms [1] reach comparable asymptotics, again unattenuated.

The contribution here is the extension to the attenuated transform (2), where the smoothness of the exponent and of the integrand must be exploited together. Specifically:

1. an exact concatenation identity (Proposition 1) for a triple of segment integrals, which makes the doubling of segment length exact and, in addition, damping: an error committed upstream is multiplied by $e^{-E} \leq 1$ at every merge;
2. identification of the quantities that admit interpolation — the exponent E rather than e^{-E} , and an exit-referenced rather than midpoint-referenced attenuated integral — with the failure mode of each alternative;
3. a sampling analysis of the three axes (direction, transverse offset, position along the ray) which fixes the level schedule and shows that the angular refinement must be placed at the finest levels, not the coarsest;
4. a measured account of accuracy and cost, and the finding that both are decided by the indexing rather than by the interpolation laid over it: a rotating ray frame, the obvious choice, needs 192 gathers per table entry in two dimensions and 3072 in three, where computing the entry from the image directly needs 256 and 768.

Section 6 then asks what the construction is worth on the hardware reconstruction actually runs on.

2 Segment integrals and the concatenation identity

The algorithm works with integrals over finite segments of a ray rather than over whole lines. Fix (θ, s) and write $f(t)$, $\mu(t)$ for the restrictions to that ray. For $a < b$ define

$$E[a, b] = \int_a^b \mu(\tau) d\tau, \quad S[a, b] = \int_a^b f(t) dt, \quad I[a, b] = \int_a^b f(t) e^{-\int_t^b \mu(\tau) d\tau} dt. \quad (7)$$

The attenuation in I is referenced to the segment's own exit point b , so every weight lies in $(0, 1]$. This choice is what makes the recursion stable (Remark 1).

Proposition 1 (Concatenation). *For any $c \in (a, b)$,*

$$E[a, b] = E[a, c] + E[c, b], \quad S[a, b] = S[a, c] + S[c, b], \quad I[a, b] = e^{-E[c, b]} I[a, c] + I[c, b]. \quad (8)$$

Proof. The first two are additivity of the integral. For the third, split at c . For $t \in [a, c]$ write $\int_t^b \mu = \int_t^c \mu + \int_c^b \mu$; the second term does not depend on t and factors out as $e^{-E[c, b]}$, leaving $I[a, c]$. For $t \in [c, b]$ the integrand is unchanged and contributes $I[c, b]$. $\square$

Equivalently, the lower-triangular matrices

$$M[a, b] = \begin{pmatrix} e^{-E[a, b]} & 0 \\ I[a, b] & 1 \end{pmatrix} \quad \text{satisfy} \quad M[a, b] = M[a, c] M[c, b], \quad (9)$$

so the triple (S, E, I) carries a semigroup structure under concatenation. Identity (8) involves no quadrature and no interpolation: given the triples of two adjacent segments, the triple of their union is obtained exactly. This is what permits the segment length to double from level to level at no cost in accuracy.

Remark 1 (Why I is referenced to the exit point). *Referencing the attenuation to the segment midpoint m , i.e. using $\tilde{I}[a, b] = \int_a^b f(t) e^{-\int_t^m \mu} dt$, gives weights as large as $e^{+E[a, m]}$, which grow without bound with segment length and attenuation strength. With the exit-referenced definition (7) all weights are in $(0, 1]$, and (8) shows that an error ε in $I[a, c]$ propagates into $I[a, b]$ as $e^{-E[c, b]} \varepsilon$. Strong attenuation therefore improves the conditioning of the recursion. This is confirmed in Section 5.1, where strong attenuation never degrades the computed transform.*

Remark 2 (Why E and not e^{-E} is interpolated). *E is an additive line integral of μ , smoothed along the ray by the averaging in (7), with derivatives bounded in terms of those of μ . Its exponential spans many orders of magnitude — in the experiments below E reaches 12, so $e^{-E} \approx 6 \times 10^{-6}$ — and has correspondingly large relative derivatives. The algorithm interpolates E and applies the exponential afterwards. This is the sense in which smoothness is used twice: once in the exponent, through E , and once in the integrand, through I .*

3 Resolution requirements

Three rules fix how densely the integrals must be computed; all three are in [5] and all three are needed to see why the number of integrals per level does not depend on the length.

Write $F(\mathbf{x}, L, \theta)$ for the integral over a segment of length L , normalised by L , so that F is the mean of the data along the segment. Perturbing the direction,

$$F(\mathbf{x}, L, \theta + \xi) = F(\mathbf{x}, L, \theta) + R, \quad |R| \leq \frac{\xi L}{4} \max |\nabla g|, \quad (10)$$

so the angular resolution needed for a given accuracy is proportional to L : *double the length, double the number of directions*. Moving the segment a distance ϵ along its own direction changes F by at most $2\epsilon \max |g|/L$, so the spacing of evaluation points along the direction is inversely proportional to L : *double the length, halve the number of positions*. Transverse to the direction no averaging occurs, so the spacing there is set by the bandwidth of the data and is the same at every level. The three together give the property the method rests on:

$$\text{number of integrals at length } L = \text{independent of } L. \quad (11)$$

Finally $\partial F/\partial L$ is $\mathcal{O}(1/L)$, so L itself need only be sampled geometrically — one level per doubling.

The same counting in d dimensions gives directions growing as $(L/h)^{d-1}$ and positions falling as h/L , so in three dimensions the table grows by two per level and the total is dominated by the top level. That is not a defect: the three-dimensional X-ray transform is a four-dimensional object, and the top level *is* the output.

4 The algorithm

4.1 Indexing

Everything turns on how segments are indexed. Let H be the data mesh and $h \leq H$ the integration mesh transverse to the sweep; their ratio

$$\sigma = H/h \quad (\geq 1) \quad (12)$$

is the single parameter controlling how finely the tables are sampled relative to the data, and is the paper's H/h . It is the main accuracy knob: the error of one interpolation is $\mathcal{O}(\sigma^{-p})$ and the work is $\mathcal{O}(\sigma^{2(d-1)})$. For the family of directions within 45° of $+x$, a segment of horizontal extent L runs from a mesh point (x_i, y_j) to $(x_i + L, y_j + mh)$ with m an *integer*. Its slope is mh/L , so directions are equispaced in $\tan \theta$ and the slope set at level k has spacing h/L_k , doubling in size exactly as (10) requires. Crucially, every segment both begins and ends on the mesh. Level k carries

$$L_k = 2^k L_0, \quad x \text{ starts spaced } L_k, \quad y \text{ starts spaced } h, \quad m \in [-R_k, R_k], \quad R_k = L_k/h, \quad (13)$$

whose product is independent of k , which is (11).

4.2 Doubling

For a level- $(k+1)$ target of extent $2L$ and rise M :

M even. The direction already exists at level k . The two halves are the level- k segments $[2i, j, M/2]$ and $[2i+1, j+M/2, M/2]$ — the second beginning exactly where the first ends, both on the mesh. No approximation whatever.

M odd. The direction is new. Interpolate the *slope index* over p neighbouring integer rises, separately for each half, at the same starting point:

$$A = \sum_a w_a T_k[2i, j, m_a], \quad B = \sum_a w_a T_k[2i+1, j+m_a, m_a], \quad (14)$$

with w_a the midpoint Lagrange weights for the half-rise $M/2$. The start row of the second half moves *with* the slope index m_a , because each direction's second half begins where its own first half ended. That is what removes any need for spatial interpolation: (14) contains only a one-dimensional interpolation in m and an integer shear in (j, m) , at $\mathcal{O}(p)$ operations per integral.

4.3 What is interpolated

Three choices, each of which the measurements in Section 5 penalise if made differently.

Normalise by arc length. A segment of extent L and rise mh has arc length $\sqrt{L^2 + (mh)^2}$, so interpolating an unnormalised integral in the slope index interpolates the arc length along with it and leaves a purely geometric error unrelated to the data. Storing mean values removes it: a constant field is then reproduced exactly.

Interpolate E , not e^{-E} , and exponentiate afterwards, for the reason given in Remark 2; and *reference I to the segment's own exit point*, for the reason given in Remark 1.

Interpolate, then concatenate. For a plain line integral the merge is an average, so it commutes with the interpolation and the order is immaterial — both readings of [5] give identical results. The attenuated merge is nonlinear in E and does not commute. Each half's triple must be interpolated first and the two concatenated afterwards by (8), which keeps the concatenation exact. Writing ℓ for the arc length of each half of the *target* segment,

$$\bar{S} = \frac{1}{2}(\bar{S}_A + \bar{S}_B), \quad \bar{E} = \frac{1}{2}(\bar{E}_A + \bar{E}_B), \quad \bar{I} = \frac{1}{2}(e^{-\bar{E}_B \ell} \bar{I}_A + \bar{I}_B). \quad (15)$$

4.4 Base level, families, and three dimensions

Only the base level touches the data: the shortest segments are evaluated by Gauss–Legendre quadrature with the fields sampled by tensor-product interpolation from the image grid. Four sweeps are needed in two dimensions and six in three — the shallow cone about each axis direction — because the attenuated transform is *directed*: the exponent is referenced to the exit end, so θ and $\theta + \pi$ differ. Each is the same code applied to a reflected or transposed image.

In three dimensions a segment carries two integer rises, and the construction is unchanged. The targets form a grid in the two slope indices, so the two-dimensional interpolation separates into two one-dimensional passes and still costs $\mathcal{O}(p)$ per entry rather than $\mathcal{O}(p^2)$; the shear becomes two successive integer gathers, one per transverse axis.

4.5 Complexity

With (11) the table size is the same at every level, there are $l \approx \log n$ levels, and (14) costs $\mathcal{O}(p)$ per integral, so in two dimensions

$$W = \mathcal{O}(p \sigma^2 N \log N). \quad (16)$$

The error of one interpolation is $\mathcal{O}((h/H)^p) = \mathcal{O}(\sigma^{-p})$; over l levels, reaching a prescribed accuracy ε requires $l \sigma^{-p} \approx \varepsilon$, so at fixed σ

$$p = \mathcal{O}(\log(1/\varepsilon) + \log \log n), \quad \text{hence} \quad W = \mathcal{O}(N \log N \log(1/\varepsilon)). \quad (17)$$

Optimising p against σ instead gives the form $\mathcal{O}(pn(\log n)^{1+2/p})$ reported in [5], minimised at $p = 2 \ln \log n$ to give $\mathcal{O}(n \log n \log \log n)$. Either way the cost per integral is a constant times the interpolation order, which is what distinguishes this indexing from one in which the ray frame rotates with the direction: there each entry would need a scattered d -dimensional resample at $\mathcal{O}(p^d)$, and in three dimensions that is more expensive than computing the entry from the image directly.

5 Numerical results

Two codes are exercised. `bdlines` reproduces [5] for plain line integrals, so that the discretisation, the merging, the interpolation and the work count can each be checked against published numbers before any of it is carried into the attenuated case; `tomogrid` is the attenuated transform on the same indexing. All numbers below are produced by committed scripts over committed data.

5.1 Exactness where the method claims it

The properties that must hold to roundoff are the sharpest test of the indexing, and each of them failed for a formulation in which the ray frame rotates with the direction.

A direction already present at the base level is merged and never interpolated, and the normalised trapezoid rule composes exactly, so the recursion must agree with the direct discretisation to roundoff: measured 1.2×10^{-16} . Constant data must be reproduced exactly at every level, which additionally tests the arc-length normalisation: measured 4.4×10^{-16} , in two and in three dimensions. With $\mu \equiv 0$ the computed I equals S to 10^{-12} ; with $f \equiv 0$ both vanish identically while E does not; and the transform is linear in f to 10^{-12} .

5.2 Reproducing Brandt and Dym

The central claim of [5] is that halving the integration mesh reduces the approximation error by 2^p . Measured over $H/h = 1, 2, 4$ on the paper’s sinusoidal target: linear doubling gives $3.8\times$ and $3.9\times$, cubic gives $16\times$ and $19\times$, quintic gives 2^6 until it reaches the discretisation floor. The discretisation errors agree with the published values to within a few per cent — 0.085% against 0.092%, 0.019% against 0.025%, 0.0062% against 0.008% — as does the near-independence of the approximation error from L .

The absolute approximation error with *linear* doubling comes out about $3.5\times$ larger than published, with the same rates. Almost all of the difference is the interpolation of the second half of each new direction, whose start row moves with the slope index; the first half contributes about five times less. Both readings of the text were implemented and give identical results, because with linear interpolation the interpolation and the averaging commute, so the published Figure 3(b) is needed to settle which four length- L integrals are averaged. The discrepancy does not affect the design criterion, which is that the approximation need only be as accurate as the discretisation it approximates: with cubic doubling the error at $H/h = 1$ is 0.015%, already well below the 0.085% discretisation error.

5.3 The attenuated transform in two dimensions

Table 1 reports the attenuated transform of a 129^2 image over all four direction families, against a dense-trapezoid evaluation of the analytic phantom, line by line on the segment endpoints the algorithm produces.

Raising the order is cheaper than refining the mesh, as (17) says it should be: at $\sigma = 1$, going from $p = 2$ to $p = 4$ reduces the error by a factor of three for a quarter more work, whereas reaching the same error by refining σ costs four times the work. At $\sigma = 1$ the sixth-order stencil is *worse* than the fourth (2.81×10^{-2} against 1.04×10^{-2}), for the reason returned to in Section 5.4; by $\sigma = 4$ the two agree to within 5%, both having reached the discretisation floor.

Table 2 refines the grid at fixed parameters, which is where (17) is tested directly.

The work ratios are 4.46, 4.41, 4.37, 4.35, against the 4 of a linear method and the 8 of direct quadrature: this is $\mathcal{O}(N \log N)$ measured, the logarithm visible as the drift above four, itself

Table 1: Attenuated transform, 129^2 image, all four direction families, against the continuum. $\sigma = H/h$; p is the slope-interpolation order. Operation counts are for the whole transform.

p	σ	$\ I - I_{\text{ref}}\ _{\infty} / \max S $	Mflop	seconds
2	1	3.06×10^{-2}	38.2	0.59
2	2	9.73×10^{-3}	142.9	1.57
2	4	3.27×10^{-3}	552.1	9.00
4	1	1.04×10^{-2}	47.7	0.38
4	2	2.26×10^{-3}	180.8	1.28
4	4	1.06×10^{-3}	703.4	9.51
6	1	2.81×10^{-2}	57.2	0.56
6	2	2.25×10^{-3}	218.7	1.63
6	4	1.01×10^{-3}	854.7	9.58

Table 2: Refinement at $p = 4$, $\sigma = 2$, all four families. “lines” is the number of attenuated line integrals produced. Each row doubles n , so N quadruples: a pure $\mathcal{O}(N)$ method would multiply the work by exactly four and an $\mathcal{O}(N^{3/2})$ one by eight. The error at $n = 513$ is measured on every fourth offset and every eighth slope; the reference costs far more than the transform it checks.

n	error	order	lines	Mflop	work ratio
33	3.67×10^{-2}	—	33 540	9.2	—
65	1.21×10^{-2}	1.60	132 612	41.0	4.46
129	2.26×10^{-3}	2.42	527 364	180.8	4.41
257	2.95×10^{-4}	2.94	2 103 300	791.1	4.37
513	1.70×10^{-5}	4.12	8 400 900	3438.4	4.35

decreasing because the level count grows only as $\log n$. The error converges at an order rising through 1.60, 2.42, 2.94 to 4.12 — the fourth order of the cubic image interpolant, reached. At $n = 513$ the transform delivers 8.4 million attenuated line integrals in 54s.

Against direct quadrature on the same lines, with the same number of samples per line as the grid has cells, the measured ratios are 2.7, 6.0 and 7.9 at $n = 65$, 129 and 257 — roughly doubling with n , which is the $n/\log n$ of Section 4.5 rather than any difference of exponents.

5.4 Three dimensions

The full four-dimensional transform of a 33^3 image — six direction families, 2.8×10^7 line integrals — takes 43s. Separating the two error sources on one family, against a reference built from the same image interpolant and against the continuum:

p	σ	e_{alg}	e_{disc}	seconds
4	1	7.08×10^{-2}	3.55×10^{-2}	6.4
4	2	7.16×10^{-3}	2.85×10^{-2}	143.6
6	1	1.04×10^{-1}	3.55×10^{-2}	6.6

Two dimensions meets the design criterion at $\sigma = 1$; three dimensions needs $\sigma = 2$, where the algorithmic error is four times *below* the discretisation error, having fallen tenfold from $\sigma = 1$. Raising the order instead does not work at $\sigma = 1$: $p = 6$ is worse than $p = 4$. On equispaced nodes the Lebesgue constant of Lagrange interpolation grows like 2^p , so the order only pays once

the table spacing is comfortably inside the field’s Nyquist rate, which at $\sigma = 1$ it is not. This is the practical content of (17): ε is bought by the order, but only above a threshold in σ that depends on the dimension.

5.5 Against a rotating ray frame

The same attenuated transform was first implemented with tables indexed by (θ, s, t) in a ray frame that rotates with the direction. Refining the direction then moves every sample point, so each entry requires a scattered d -dimensional resample: 192 gathers per entry in two dimensions and 3072 in three, against $2gp_{\text{img}}^d = 256$ and 768 to compute the entry from the image directly. In two dimensions that is marginally profitable; in three it is four times more expensive than not using the hierarchy at all, and the crossover against direct evaluation sits near $n \approx 200$ rather than $n = 65$. Measured, the 129^2 transform took 15.4s for a quarter of the directions, against 0.63s here for all of them. The lesson is that the cost and the error budget of this method are both decided by the indexing, not by the interpolation machinery laid over it.

6 Suitability for GPU hardware

Reconstruction runs on GPUs, so the question is not only how much arithmetic the recursion saves, but how much of that saving survives on hardware whose arithmetic is its cheapest resource. This section answers it from operation and traffic counts.

Nothing in this section was measured on a device. The counts are computed from array shapes and the stencil width, and can be checked against the source. The device figures divide those counts by published peak bandwidth and peak arithmetic rates. They are lower bounds on the time a good implementation would need, not measurements of one.

6.1 The shape of a doubling

A doubling is three operations on whole arrays of shape $(n_x, n_y, n_{\text{slope}})$, one array for each of S , E and I .

1. *An integer shear*, $G[i, j, m] = T[i, j + m, m]$, zero off the mesh. This is where the second half of each segment is read: it starts where the first half ended, so its transverse index moves with the slope index. The indices are known before the kernel runs. At fixed m the operation is a shift of a contiguous plane, so the reads coalesce.
2. *A p -tap weighted sum along the slope axis*, for the directions that do not exist one level down. Away from the two ends of the slope range the offsets and the weights are the same for every target. For $p = 4$ they are $(-1, 0, 1, 2)$ and $(-\frac{1}{16}, \frac{9}{16}, \frac{9}{16}, -\frac{1}{16})$. The interior is therefore a constant-coefficient convolution, and only $\mathcal{O}(p)$ columns at each end are one-sided.
3. *A pointwise merge*, (15): eight flops and one exponential per entry.

Nothing scatters. Nothing needs an atomic. Nothing branches on a value, and no index is computed from one. The only loops are over the p taps, and p is fixed before the kernels run. A device implementation is the same source with a different array namespace, which is how the reference implementation is written.

Table 3: Counted from array shapes, all four direction families, $p = 4$, $\sigma = 2$, double precision. “base” and “recursion” are the two halves of the algorithm; f/B is flops per byte of traffic, with one exponential charged twenty flops. “live” is the largest working set, three tables of three quantities. No entry is a measurement.

n	lines	base GB	recursion GB	base f/B	recursion f/B	live GB
65	132,612	0.005	0.049	35.7	0.70	0.011
129	527,364	0.018	0.235	35.7	0.71	0.043
257	2,103,300	0.074	1.090	35.7	0.71	0.170
513	8,400,900	0.294	4.962	35.7	0.71	0.680

6.2 The parallelism does not thin out with level

This is what separates the construction from a multigrid cycle. In a cycle the coarse levels hold too few points to fill a device, and the coarsest levels are latency-bound; much of the literature on multigrid for GPUs is about that problem. Here the table is the same size at every level, by construction: the number of segment start positions halves exactly as the number of directions doubles (Section 4.1). At $n = 513$, $\sigma = 2$ the nine levels hold between 2.10 and 2.36 million entries per quantity per family, a spread of 12%. Every level therefore presents about 2.5×10^7 independent outputs across the three quantities and the four families, and the last presents as many as the first. Occupancy does not collapse anywhere in the recursion.

6.3 The adjoint keeps the same access pattern

Every iterative method in Section 1.2 applies $\mathcal{R}_\mu^*$ as often as $\mathcal{R}_\mu$. On a GPU the transpose of a gather is usually a scatter, which means atomics and contention; this is the standard reason backprojection is harder to parallelise than projection. It does not happen here. With μ fixed, $\mathcal{R}_\mu$ is linear in f and the merge coefficients $e^{-E\ell}$ are computed once in the forward sweep. The transpose of an integer shear is the opposite integer shear. The transpose of a banded operator is banded, so the anteropolar dual to the slope interpolation is again a gather, with $\mathcal{O}(p)$ taps read through the transposed stencil. The adjoint is thus the same three kernels with the same access pattern, at the same cost.

6.4 Where the recursion sits on the roofline

Table 3 counts the traffic. Two figures matter. The base level interpolates both fields at every Gauss point of every short segment, out of an image of a few megabytes that stays in cache; it runs at 35.7 flops per byte and is arithmetic-dense. The recursion is the opposite. A doubling reads three tables, writes three tables, and performs about fifty flops on each entry, so it runs at 0.71 flops per byte. Current parts balance at 4.8 (A100) to 10.1 (H100) flops per byte in double precision, so the recursion is bandwidth-bound by an order of magnitude and its time is its traffic divided by memory bandwidth.

The comparison against direct quadrature therefore does not reduce to the arithmetic ratio. Direct quadrature of the same lines reads an image that fits in cache, so it is arithmetic-bound; the recursion streams tables, so it is bandwidth-bound. The two are limited by different resources, and the resource the recursion saves is the one the machine has most of. Table 4 gives both rooflines at $n = 513$.

The ratio doubles with every refinement, as the $\mathcal{O}(n/\log n)$ arithmetic ratio requires, but it is

Table 4: Rooflines at $n = 513$, 8.4×10^6 line integrals, four families: counted traffic and arithmetic divided by published peak figures. “ML” is the algorithm of this paper, “direct” is quadrature of the same lines. The bound is memory (m) or arithmetic (c). Lower bounds, not measurements.

device	balance (f/B)	ML			direct	ratio
A100 80GB SXM, fp64	4.8	2.6 ms	(m)		122 ms (c)	47
A100 80GB SXM, fp32	9.6	1.3 ms	(m)		53.7 ms (c)	42
H100 SXM5, fp64	10.1	1.6 ms	(m)		34.9 ms (c)	22
H100 SXM5, fp32	20.0	0.8 ms	(m)		15.6 ms (c)	20
RTX 4090, fp64	1.3	10.9 ms	(c)		919 ms (c)	84
RTX 4090, fp32	81.9	2.6 ms	(m)		12.7 ms (c)	5

smaller than that ratio and it depends on the machine. The last row is the instructive one. A consumer card pairs 82.6 Tflop/s of single precision with 1 TB/s of bandwidth, a balance of 82 flops per byte, so brute force is almost free there and the recursion wins only by five. Stated generally: the construction trades arithmetic for memory traffic, and GPU hardware has been moving the other way for a decade. That is an argument for fusing successive doublings, so that a tile of the table is refined twice while resident in registers, which trades flops back for traffic. At 0.71 flops per byte there is room for it.

6.5 Single precision suffices

This matters because a consumer card runs double precision at a sixty-fourth of its single-precision rate. The merge weight $e^{-E\ell}$ lies in $(0, 1]$, so the recursion damps roundoff rather than amplifying it, and the number of levels should not enter. Running the whole pipeline in single precision against the double-precision result gives a relative deviation of 2.6×10^{-7} at five doublings, 2.6×10^{-7} at six and 3.1×10^{-7} at seven: flat in the level count, and two to three orders of magnitude below the discretisation error at those grids. Single precision halves the traffic and, by Table 4, halves the time.

6.6 What the construction does not use, and what limits it

It does not use tensor cores, and should not. The slope interpolation is a banded matrix applied along one axis, which invites a dense product; the arithmetic forbids it. The matrix has $p = 4$ nonzeros per row and n_{slope} reaches 2049, so a dense product costs some 500 times the arithmetic, against the factor of eight that reduced-precision tensor cores return. Blocking the slope axis into tiles of 32 softens this to 8.75 times the arithmetic for eight times the rate — a wash, at reduced precision, on a kernel that is bandwidth-bound in any case. The tensor-product structure does pay, but at the level of the algorithm: the two slope axes in three dimensions are refined one at a time, at $\mathcal{O}(p)$ per entry instead of $\mathcal{O}(p^2)$, and the base level is a tensor-product interpolation, which is the arithmetic-dense work that would keep the units busy while the recursion waits on memory.

What limits the construction is memory, and only in three dimensions. In two dimensions the working set at $n = 513$ is 0.68 GB in double precision and half that in single. In three dimensions the output is itself four-dimensional: one direction family of six at $n = 65$, $\sigma = 2$ holds 79 GB in double precision, which is the capacity of a large device, and 40 GB in single. Three-dimensional problems at clinical resolution therefore have to be blocked over direction families and over slabs of the transverse plane — which the construction permits, since families are independent and the transverse index is never interpolated — or streamed. We have not implemented either.

7 Discussion

Limitations. Four, stated precisely. (i) Directions come out equispaced in slope rather than in angle, and the transverse spacing between neighbouring lines therefore varies with angle, by $\sqrt{2}$ across a quadrant; [5] notes both an equispaced-grid variant and the simpler remedy of interpolating to a uniform grid at the final stage only. (ii) On rays of very large total attenuation the true signal underflows, and what the method controls there is a small error *absolute* to $\max |S|$, not a small relative error; carrying the mean survival weight I/S instead of I would address this at the cost of an indeterminate form wherever a segment misses the support. (iii) Zero extension outside the tables is exact only for compactly supported fields. (iv) The order p buys accuracy only above a threshold in σ that grows with dimension, because equispaced Lagrange stencils have Lebesgue constants growing like 2^p ; better-conditioned transfers would remove the threshold and are the natural way to make (17) hold uniformly.

The adjoint, and the inverse problem. The adjoint $\mathcal{R}_\mu^*$, which every method in Section 1.2 applies once per sweep alongside the forward operator, is the transpose of the construction step by step: the concatenation transposes to a scatter and the slope interpolation to antepolation, in the sense of [6]. Because the doubling here is an indexed multiply-add rather than a scattered resample, its transpose is again an indexed multiply-add, and the adjoint costs what the forward operator costs. That is what makes the present formulation, rather than the rotating-frame one, a usable basis for the reconstruction itself.

Beyond that lies the inverse problem sketched in Section 1.2: the linear solve for f with μ known, and then the joint estimation of (f, μ) , in which μ enters nonlinearly because it sits in the exponent. A row-action relaxation of (5) smooths the error components the fine grid resolves and leaves the rest to coarser grids, so a full multigrid cycle over the image hierarchy is the natural outer solver, and a full approximation scheme [3] the natural treatment of the nonlinearity. The level structure built here — lengths paired with angular resolutions — is the hierarchy such a solver would traverse, which is what makes evaluation and solver composable. [5] notes that its own construction can in principle be inverted directly; whether that carries over to the attenuated case is open.

8 Conclusion

The attenuated Radon transform admits an $\mathcal{O}(N \log N)$ multilevel evaluation. The construction rests on an exact concatenation identity for the triple (S, E, I) , on interpolating the exponent rather than its exponential and the exit-referenced rather than the midpoint-referenced attenuated integral, and on a level schedule that refines the angular grid at the finest levels where segments are short. On a battery of attenuation fields the error the schedule introduces stays an order of magnitude below the discretisation error of the grid and converges with it, at a cost already several times below direct evaluation at moderate resolution and improving as $\mathcal{O}(\sqrt{N}/\log N)$.

Code

The reference implementation, the reproduction of [5], the test suite and the scripts producing every number above are at [12]; its README lists the commands.

References

- [1] A. Averbuch, R. R. Coifman, D. L. Donoho, M. Israeli, and Y. Shkolnisky. A framework for discrete integral transformations I — the pseudopolar Fourier transform. *SIAM Journal on Scientific Computing*, 30(2):764–784, 2008. doi: 10.1137/060650283.
- [2] Samit Basu and Yoram Bresler. $O(N^2 \log_2 N)$ filtered backprojection reconstruction algorithm for tomography. *IEEE Transactions on Image Processing*, 9(10):1760–1772, 2000. doi: 10.1109/83.869187.
- [3] Achi Brandt. Multi-level adaptive solutions to boundary-value problems. *Mathematics of Computation*, 31(138):333–390, 1977. doi: 10.1090/S0025-5718-1977-0431719-X.
- [4] Achi Brandt. Multilevel computations of integral transforms and particle interactions with oscillatory kernels. *Computer Physics Communications*, 65(1–3):24–38, 1991. doi: 10.1016/0010-4655(91)90151-A.
- [5] Achi Brandt and Jordan Dym. Fast calculation of multiple line integrals. *SIAM Journal on Scientific Computing*, 20(4):1417–1429, 1999.
- [6] Achi Brandt and A. A. Lubrecht. Multilevel matrix multiplication and fast solution of integral equations. *Journal of Computational Physics*, 90(2):348–370, 1990. doi: 10.1016/0021-9991(90)90171-V.
- [7] Achi Brandt, Jordan Mann, Matvei Brodski, and Meirav Galun. A fast and accurate multilevel inversion of the Radon transform. *SIAM Journal on Applied Mathematics*, 60(2):437–462, 2000.
- [8] Richard Gordon, Robert Bender, and Gabor T. Herman. Algebraic reconstruction techniques (ART) for three-dimensional electron microscopy and X-ray photography. *Journal of Theoretical Biology*, 29(3):471–481, 1970. doi: 10.1016/0022-5193(70)90109-8.
- [9] Leslie Greengard and Vladimir Rokhlin. A fast algorithm for particle simulations. *Journal of Computational Physics*, 73(2):325–348, 1987.
- [10] H. Malcolm Hudson and Richard S. Larkin. Accelerated image reconstruction using ordered subsets of projection data. *IEEE Transactions on Medical Imaging*, 13(4):601–609, 1994. doi: 10.1109/42.363108.
- [11] Kyung Sang Kim and Jong Chul Ye. Fully 3D iterative scatter-corrected OSEM for HRRT PET using a GPU. *Physics in Medicine and Biology*, 56(15):4991–5009, 2011. doi: 10.1088/0031-9155/56/15/021.
- [12] Oren E. Livne and Achi Brandt. **tomogrid**: multilevel evaluation of the attenuated Radon transform. <https://github.com/orenlivne/tomogrid>, 2026. Version 0.1.0. Reference implementation, test suite and the scripts producing every table and figure of this paper.
- [13] Frank Natterer. *The Mathematics of Computerized Tomography*, volume 32 of *Classics in Applied Mathematics*. SIAM, Philadelphia, 2001.
- [14] Roman G. Novikov. An inversion formula for the attenuated X-ray transformation. *Arkiv för Matematik*, 40(1):145–167, 2002. doi: 10.1007/BF02384507.

- [15] Georg Schramm and Kris Thielemans. PARALLELPROJ — an open-source framework for fast calculation of projections in tomography. *Frontiers in Nuclear Medicine*, 3, 2024. doi: 10.3389/fnume.2023.1324562.
- [16] L. A. Shepp and Y. Vardi. Maximum likelihood reconstruction for emission tomography. *IEEE Transactions on Medical Imaging*, 1(2):113–122, 1982. Printed as volume MI-1.